

WEEK-05

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int minDistance(int dist[], int visited[], int n) {
    int min = INT_MAX, min_index = -1;

    for (int i = 0; i < n; i++) {
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            min_index = i;
        }
    }
    return min_index;
}

void dijkstra(int graph[MAX][MAX], int n, int src) {
    int dist[MAX], visited[MAX];

    // Initialization
    for (int i = 0; i < n; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }

    dist[src] = 0;

    // Main loop
    for (int count = 0; count < n - 1; count++) {
        int u = minDistance(dist, visited, n);
        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (!visited[v] &&
                graph[u][v] != 0 &&
                dist[u] != INT_MAX &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
}
```

```

    }
}

// Output
printf("\nVertex\tDistance from Source\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\n", i, dist[i]);
}
}

int main() {
    int n, graph[MAX][MAX], src;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (0 if no edge):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter source vertex (0 to %d): ", n - 1);
    scanf("%d", &src);

    dijkstra(graph, n, src);

    return 0;
}

```

Output:

```

"C:\Users\BMSCECSE\Desktop"
Enter number of vertices: 4
Enter adjacency matrix (0 if no edge):
1 2 3 5
2 6 3 4
1 1 0 1
4 5 2 3
Enter source vertex (0 to 3): 1

Vertex  Distance from Source
0       2
1       0
2       3
3       4

```